

React `useEffect` — Cheatsheet per la lezione

Lezione 816 — React: `useEffect`

Punto del corso: dopo JSX, componenti, iterazioni, props, `useState` e data binding; prima di data fetching, form/POST, React Router e Context API.

1. L'idea fondamentale

Un componente React ha principalmente due responsabilità:

1. **renderizzare l'interfaccia** a partire da props e state;
2. **sincronizzarsi con sistemi esterni** tramite gli effetti collaterali.

`useEffect` è un Hook che permette di eseguire del codice **dopo che React ha aggiornato il DOM**.

```
import { useEffect } from 'react';

useEffect(() => {
  // effetto collaterale
}, [dipendenze]);
```

Un **effetto collaterale** è un'operazione che va oltre il semplice calcolo del JSX, per esempio:

- modificare `document.title`;
- usare timer come `setInterval` e `setTimeout`;
- aggiungere un event listener;
- leggere o scrivere nel `localStorage`;
- eseguire una chiamata HTTP;
- sincronizzarsi con una libreria o un servizio esterno.

Il render, invece, dovrebbe rimanere **puro**: con gli stessi dati in ingresso deve limitarsi a calcolare lo stesso JSX, senza modificare ciò che si trova fuori dal componente.

2. Le tre forme principali

A. Senza array delle dipendenze

```
useEffect(() => {
  console.log('Eseguito dopo ogni render');
});
```

L'effetto viene eseguito dopo il primo render e dopo **ogni aggiornamento** del componente.

Attenzione: se al suo interno aggiorni uno state a ogni esecuzione, puoi creare un loop infinito.

B. Con array vuoto

```
useEffect(() => {  
  console.log('Componente montato');  
}, []);
```

L'effetto viene eseguito dopo il primo render del componente. È la forma che più assomiglia al vecchio `componentDidMount` dei class component.

C. Con dipendenze

```
useEffect(() => {  
  console.log(`Il filtro ora è: ${filter}`);  
}, [filter]);
```

L'effetto viene eseguito:

- dopo il primo render;
- ogni volta che almeno una dipendenza cambia.

React confronta ogni dipendenza con il valore del render precedente. Per questo bisogna inserire nell'array tutti i valori reattivi letti dall'effetto: props, state e variabili o funzioni dichiarate nel componente.

3. Component Life Cycle

Nel linguaggio del ciclo di vita, un componente può attraversare tre fasi:

Fase	Significato	Relazione con <code>useEffect</code>
Mount	il componente viene inserito nella pagina	l'effetto viene eseguito per la prima volta
Update	cambiano props o state e il componente viene renderizzato di nuovo	l'effetto si riesegue se sono cambiate le sue dipendenze
Unmount	il componente viene rimosso dalla pagina	React esegue l'eventuale funzione di cleanup

Sequenza mentale semplificata:

```
render → aggiornamento del DOM → effect
```

dipendenza cambiata:

```
render → aggiornamento del DOM → cleanup precedente → nuovo effect
```

```
unmount:  
cleanup finale
```

Nei componenti funzionali non serve pensare a tre metodi separati: uno stesso `useEffect` descrive come avviare e come fermare una sincronizzazione.

4. Cleanup: pulire un effetto

Un effetto può restituire una funzione di **cleanup**:

```
useEffect(() => {  
  const intervalId = setInterval(() => {  
    console.log('tick');  
  }, 1000);  
  
  return () => {  
    clearInterval(intervalId);  
  };  
}, []);
```

La cleanup viene eseguita:

- prima che lo stesso effetto venga eseguito di nuovo con nuove dipendenze;
- quando il componente viene smontato.

Serve per evitare timer duplicati, listener rimasti attivi, connessioni aperte e memory leak.

Altro esempio:

```
useEffect(() => {  
  function handleResize() {  
    console.log(window.innerWidth);  
  }  
  
  window.addEventListener('resize', handleResize);  
  
  return () => {  
    window.removeEventListener('resize', handleResize);  
  };  
}, []);
```

La stessa funzione `handleResize` viene passata sia ad `addEventListener` sia a `removeEventListener`.

5. Esempio: filtro di una todolist

Versione didattica con `useEffect`

Questa versione mostra come aggiornare un risultato quando cambiano `todos` o `filter`:

```
import { useEffect, useState } from 'react';

const initialTodos = [
  { id: 1, text: 'Studiare useEffect', completed: true },
  { id: 2, text: 'Fare gli esercizi', completed: false },
  { id: 3, text: 'Ripassare le props', completed: true },
];

function TodoList() {
  const [todos, setTodos] = useState(initialTodos);
  const [filter, setFilter] = useState('all');
  const [filteredTodos, setFilteredTodos] = useState(initialTodos);

  useEffect(() => {
    if (filter === 'completed') {
      setFilteredTodos(todos.filter((todo) => todo.completed));
    } else if (filter === 'active') {
      setFilteredTodos(todos.filter((todo) => !todo.completed));
    } else {
      setFilteredTodos(todos);
    }
  }, [todos, filter]);

  return (
    <>
      <select
        value={filter}
        onChange={(event) => setFilter(event.target.value)}
      >
        <option value='all'>Tutte</option>
        <option value='active'>Da fare</option>
        <option value='completed'>Completate</option>
      </select>

      <ul>
        {filteredTodos.map((todo) => (
          <li key={todo.id}>{todo.text}</li>
        ))}
      </ul>
    </>
  );
}
```

Come spiegarla:

- `filter` contiene il filtro selezionato;
- `filteredTodos` contiene le todo da mostrare;

- l'effetto dipende da `todos` e `filter`, perché legge entrambi;
- quando uno dei due cambia, React renderizza e poi esegue l'effetto;
- l'effetto aggiorna `filteredTodos`, causando un altro render.

Nota importante: qui l'effetto non è strettamente necessario

`filteredTodos` è un dato **derivato** da state già esistente. Non c'è un sistema esterno da sincronizzare, quindi normalmente è più semplice calcolarlo direttamente durante il render:

```
const filteredTodos = todos.filter((todo) => {  
  if (filter === 'completed') return todo.completed;  
  if (filter === 'active') return !todo.completed;  
  return true;  
});
```

Vantaggi:

- uno state in meno;
- un render in meno;
- nessun rischio che `filteredTodos` rimanga fuori sincronia;
- codice più facile da leggere.

La versione con `useEffect` è utile per capire il meccanismo della lezione. La regola pratica da ricordare è:

Se un valore si può calcolare da props o state durante il render, spesso non serve né un altro state né un effect.

6. Errori comuni

Loop infinito

```
useEffect(() => {  
  setCount(count + 1);  
});
```

Succede questo:

effect → setCount → nuovo render → effect → setCount → ...

Anche questa versione crea un loop, perché l'effetto modifica la propria dipendenza:

```
useEffect(() => {  
  setCount(count + 1);
```

```
}, [count]);
```

Dipendenza mancante

```
useEffect(() => {  
  document.title = `Ciao ${name}`;  
}, []); // manca name
```

Se **name** cambia, il titolo non viene aggiornato. La dipendenza corretta è:

```
useEffect(() => {  
  document.title = `Ciao ${name}`;  
}, [name]);
```

Inserire l'effetto dentro una condizione

```
if (isLoggedIn) {  
  useEffect(() => {  
    // ...  
  }, []);  
}
```

È sbagliato. Gli Hook vanno chiamati sempre al livello principale del componente, nello stesso ordine a ogni render. La condizione va messa **dentro** l'effetto:

```
useEffect(() => {  
  if (!isLoggedIn) return;  
  
  // ...  
}, [isLoggedIn]);
```

Dimenticare la cleanup

```
useEffect(() => {  
  setInterval(() => console.log('tick'), 1000);  
}, []);
```

Il timer continua a esistere anche quando il componente non serve più. Bisogna salvare l'id e chiamare **clearInterval** nella cleanup.

Usare **useEffect** per gestire un click

Se il codice deve partire perché l'utente ha cliccato, di solito appartiene direttamente all'event handler:

```
function handleSubmit(event) {  
  event.preventDefault();  
  console.log('Form inviato');  
}
```

Non serve creare uno state solo per far reagire un effetto al click.

7. Props, state ed effect: cosa provoca cosa?

- Cambiare uno **state** tramite il suo setter richiede un nuovo render.
- Ricevere nuove **props** può causare un nuovo render.
- Un nuovo render **non significa automaticamente** che ogni effect venga rieseguito: dipende dall'array delle dipendenze.
- Un effect viene eseguito dopo il render, non durante.
- Se un effect aggiorna uno state, richiede un ulteriore render.
- Modificare direttamente una normale variabile non provoca un render.

```
function Profile({ username }) {  
  const [visits, setVisits] = useState(0);  
  
  useEffect(() => {  
    document.title = `${username}: ${visits} visite`;  
  }, [username, visits]);  
  
  return <button onClick={() => setVisits(visits + 1)}>Visita</button>;  
}
```

L'effetto legge `username` e `visits`, quindi entrambi devono comparire nelle dipendenze.

8. `useEffect` e modalità Strict di React

Durante lo sviluppo, con `<StrictMode>`, React può eseguire una sequenza aggiuntiva di setup e cleanup per controllare che l'effetto sia scritto correttamente.

Perciò potresti vedere un `console.log` due volte al mount. Non significa necessariamente che `useEffect` sia rotto. L'effetto deve essere progettato in modo che la cleanup annulli correttamente ciò che il setup ha fatto.

```
useEffect(() => {  
  console.log('setup');  
  
  return () => {  
    console.log('cleanup');  
  };  
}, []);
```

```
};  
}, []);
```

In sviluppo potresti osservare:

```
setup → cleanup → setup
```

Questo controllo extra non avviene nello stesso modo nella build di produzione.

9. Domande probabili dell'insegnante

Che cos'è `useEffect`?

È un Hook di React usato per eseguire effetti collaterali dopo il render e per sincronizzare un componente con qualcosa di esterno a React.

Quando viene eseguito?

Dopo che React ha aggiornato il DOM. La frequenza dipende dall'array: dopo ogni render senza array, al mount con `[]`, al mount e al cambio delle dipendenze con `[a, b]`.

A cosa serve l'array delle dipendenze?

Indica quali valori usati dall'effetto devono farlo sincronizzare nuovamente quando cambiano.

Qual è la differenza tra `[]` e nessun array?

Con `[]` l'effetto parte solo al mount, salvo i controlli extra di Strict Mode in sviluppo. Senza array parte dopo ogni render.

Che cos'è la cleanup?

È la funzione restituita dall'effetto. Annulla la sincronizzazione precedente, per esempio rimuove un listener o ferma un timer. React la chiama prima di rieseguire l'effetto e all'unmount.

Posso rendere `useEffect` direttamente `async`?

No: la callback deve restituire `undefined` oppure una funzione di cleanup, non una Promise. Si definisce una funzione asincrona interna:

```
useEffect(() => {  
  async function loadData() {  
    // const response = await fetch(...);  
  }  
})
```



```
loadData();  
}, []);
```

Perché un effect può creare un loop infinito?

Perché aggiorna uno state, l'aggiornamento causa un render e il nuovo render fa ripartire l'effetto. Accade soprattutto senza dipendenze corrette o quando l'effetto aggiorna sempre un valore da cui dipende.

useEffect viene eseguito durante il render?

No. Il componente viene prima renderizzato, React aggiorna il DOM e poi esegue l'effetto.

Ogni cambiamento di una variabile fa ripartire l'effetto?

No. React rivaluta le dipendenze in occasione di un render. Una normale variabile non provoca da sola un nuovo render; props e state partecipano al flusso reattivo.

Quando non dovrei usare useEffect?

Quando posso calcolare un valore direttamente da props e state oppure quando il codice appartiene a un event handler. `useEffect` serve soprattutto per la sincronizzazione con sistemi esterni.

useEffect sostituisce il ciclo di vita?

Nei componenti funzionali permette di gestire mount, aggiornamenti e unmount, ma è meglio pensarlo come una sincronizzazione: setup dell'effetto e relativa cleanup.

10. Vero o falso — ripasso rapido

1. `useEffect(() => {}, [])` viene eseguito dopo ogni render. **Falso**.
2. Un effect può restituire una funzione. **Vero**, è la cleanup.
3. Gli Hook si possono chiamare dentro un `if`. **Falso**.
4. Aggiornare uno state può causare un nuovo render. **Vero**.
5. Tutti i calcoli derivati da uno state richiedono `useEffect`. **Falso**.
6. Senza array delle dipendenze, l'effetto parte dopo ogni render. **Vero**.
7. La cleanup viene chiamata soltanto all'unmount. **Falso**, anche prima della successiva esecuzione dell'effetto.
8. Un event listener aggiunto in un effect dovrebbe normalmente essere rimosso nella cleanup. **Vero**.
9. La callback di `useEffect` dovrebbe essere dichiarata direttamente `async`. **Falso**.
10. In Strict Mode un effect può sembrare eseguito due volte durante lo sviluppo. **Vero**.

11. Mini esercizi da saper svolgere

Aggiornare il titolo della pagina

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Conteggio: ${count}`;
  }, [count]);

  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

Domanda: perché `[count]`? Perché l'effetto legge `count` e deve risincronizzare il titolo quando cambia.

Timer con cleanup

```
function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const intervalId = setInterval(() => {
      setSeconds((currentSeconds) => currentSeconds + 1);
    }, 1000);

    return () => clearInterval(intervalId);
  }, []);

  return <p>Secondi: {seconds}</p>;
}
```

Qui si usa la forma funzionale del setter:

```
setSeconds((currentSeconds) => currentSeconds + 1);
```

In questo modo l'aggiornamento parte sempre dal valore più recente e l'effetto non deve leggere `seconds`.

Eseguire una sincronizzazione quando cambia una prop

```
function UserStatus({ userId }) {
  useEffect(() => {
    console.log(`Sincronizzo l'utente ${userId}`);

    return () => {
      console.log(`Interrompo la sincronizzazione di ${userId}`);
    };
  }, [userId]);
}
```

```
return <p>Utente: {userId}</p>;  
}
```

Quando `userId` cambia, React esegue prima la cleanup relativa al vecchio id e poi il setup con il nuovo id.

12. Formula mentale per decidere

Prima di scrivere un effect, chiediti:

1. **Sto reagendo a un'azione precisa dell'utente?** Usa un event handler.
2. **Sto soltanto calcolando un valore da props o state?** Calcolalo durante il render.
3. **Devo sincronizzare il componente con qualcosa di esterno?** Usa `useEffect`.
4. **La sincronizzazione va annullata?** Restituisci una cleanup.
5. **Quali valori reattivi leggo?** Inseriscili nelle dipendenze.

La frase da ricordare:

Il render descrive la UI; `useEffect` sincronizza la UI con il mondo esterno.